
Oliver Krauss

Database Access via PHP

RDB | MTD.ba VZ SS26

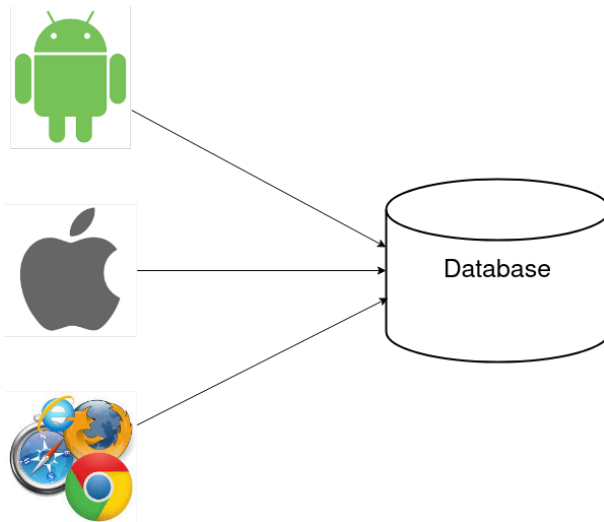
Hagenberg 29.04.2026

HAGENBERG | LINZ | STEYR | WELS

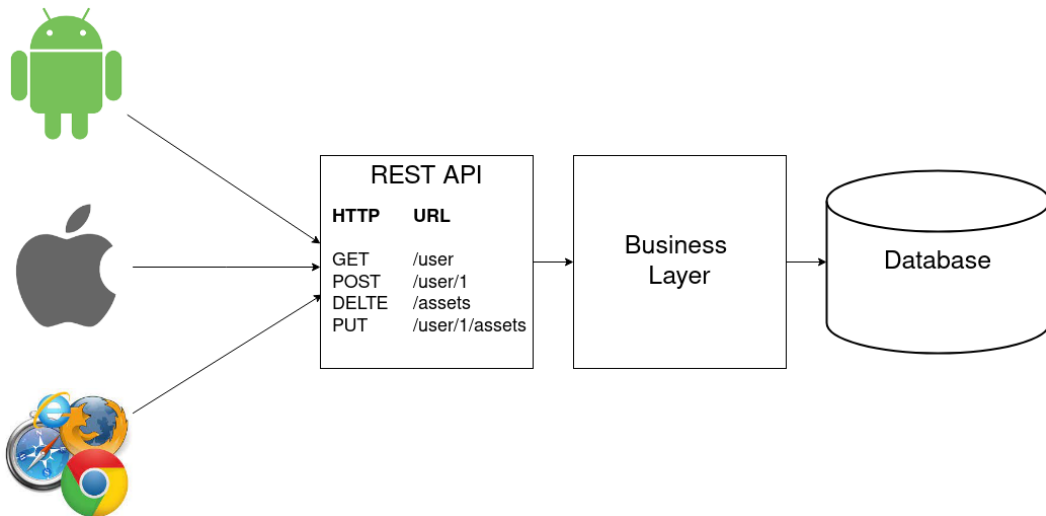


UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Database Access



Layered Database Access



Three layer architecture

Presentation Layer

- User interface
- Data presented to users

Three layer architecture

Presentation Layer

- User interface
- Data presented to users

Business Layer

- Functionality of software
- Integrity checks
- ...

Three layer architecture

Presentation Layer

- User interface
- Data presented to users

Business Layer

- Functionality of software
- Integrity checks
- ...

Data Access Layer

- Manage DB access
- Create
- Read
- Update
- Delete

Data Access Layer (DAL)

Separation of Concerns separating data access from business logic makes it easier to manage according to the *single responsibility principle*.

Abstraction Hides SQL queries from application logic

Centralized Data Access All connections, transactions, error handling managed consistently in application

Security preventing SQL injection and attacks through prepared statements

PHP Data Objects (PDO) I

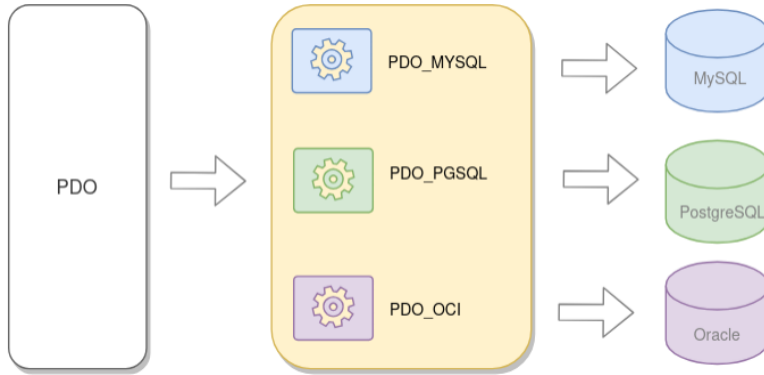


Figure: © 2023 - by phptutorial.net

PHP Data Objects (PDO) II

- Create database connections
- Execute queries
- Call stored procedures
- Perform transactions
- Handle exceptions
- For many databases

Data Source Names (DSN)

A Data Source Name (DSN) is a string that describes the connection to a database

Connection Type

/ Driver

Password

Port

mysql://dbuser:password@localhost:port/TheDatabase

Username

Server
/ Driver

Database
Name

Creating a PDO in PHP

```
1  $host = "localhost";
2  $db = "MyDatabase";
3  $dsn = "mysql:host=$host;dbname=$db";
4
5  $username = "root";
6  $password = "password";
7  $pdo = new PDO($dsn, $username, $password);
```

Creating a PDO with exception handling

```
1  $host = "localhost";
2  $db = "MyDatabase";
3  $dsn = "mysql:host=$host;dbname=$db";
4
5  $username = "root";
6  $password = "password";
7
8  try {
9      $pdo = new PDO($dsn, $username, $password);
10     echo "Connected successfully!";
11 } catch (PDOException $e) {
12     echo "Connection failed: " . $e->getMessage();
13 }
```

PDO Queries

- Now **\$pdo** allows interaction with the database
- SQL queries will be returned as *associative array*
- Arrays in PHP
 - Indexed Arrays
 - Associative Arrays
 - Multidimensional Arrays

Indexed Arrays

```
1  $students = array("James", "Hermione", "Frodo");
2
3  // Accessing elements by index
4  echo $students[0]; // Outputs: James
5  echo $students[1]; // Outputs: Hermione
6  echo $students[2]; // Outputs: Frodo
7
8  // Modifying an element
9  $students[1] = "Hermione";
10 echo $students[1]; // Outputs: Hermione
```

Associative Arrays

```
1  $student = array("name" => "Hermione", "department" => "IM", "avg_grade" =>
    3.95);
2
3  // Accessing elements by key
4  echo $student["name"]; // Outputs: Hermione
5  echo $student["department"]; // Outputs: IM
6  echo $student["avg_grade"]; // Outputs: 3.95
7
8  // Modifying an element
9  $student["avg_grade"] = 1.1;
10 echo $student["avg_grade"]; // Outputs: 1.1
```

Multidimensional Arrays

```
1 $students = array(  
2     array("name" => "Hermione", "department" => "IM", "avg_grade" => 3.95),  
3     array("name" => "Luke", "department" => "SE", "avg_grade" => 2.85),  
4     array("name" => "Indiana", "department" => "MTD", "avg_grade" => 1.95),  
5 );  
6  
7 // Accessing elements by key  
8 echo $students[0]["name"]; // Outputs: Hermione  
9 echo $students[1]["avg_grade"]; // Outputs: 2.85  
10  
11 // Modifying an element  
12 $students[0]["avg_grade"] = 1.1;  
13 echo $students[0]["avg_grade"]; // Outputs: 1.1
```


Queries in PHP

```
SELECT Name, Avg_Grade FROM Students
```

```
1  $students = [  
2      0 => ['Name' => 'Hermione Granger',  
3           'Avg_Grade' => 3.95],  
4      1 => ['Name' => 'Luke Skywalker',  
5           'Avg_Grade' => 2.85],  
6      2 => ['Name' => 'Indiana Jones',  
7           'Avg_Grade' => 1.95]  
8  ];
```

Processing a query

```
1 $sql = "SELECT Name, Avg_Grade FROM Students;";
2 foreach ($pdo->query($sql) as $row)
3 {
4     echo $row['Name']."<br />";
5     echo $row['Avg_Grade'];
6 }
```

SQL Injection

A database can be exploited if the users can submit any code that they want.

SQL Injection

A database can be exploited if the users can submit any code that they want.

Access to hidden data:

```
1  $query = "1 OR 1=1";  
2  $sql = "SELECT Name, Avg_Grade FROM Students WHERE StudentId =$query;";  
3  $result = $pdo->query($sql);
```

SQL Injection

A database can be exploited if the users can submit any code that they want.

Access to hidden data:

```
1  $query = "1 OR 1=1";  
2  $sql = "SELECT Name, Avg_Grade FROM Students WHERE StudentId =$query;";  
3  $result = $pdo->query($sql);
```

Modify or destroy data:

```
1  $query = "1; DROP TABLE Students";  
2  $sql = "SELECT Name, Avg_Grade FROM Students WHERE StudentId =$query;";  
3  $result = $pdo->query($sql);
```

Prepared Statements

- Feature in SQL databases to execute the same SQL statement multiple times
- Statement is pre-compiled by database
- Provides **security** as user inputs can only be data, never code
- High **efficiency** since optimization overhead only occurs once

Prepared Statement Phases

PREPARE the statement with variables represented by placeholders

Prepared Statement Phases

PREPARE the statement with variables represented by placeholders

SET Declare variables in the statement

Prepared Statement Phases

PREPARE the statement with variables represented by placeholders

SET Declare variables in the statement

EXECUTE Pass variables to statement and run it

Prepared Statement Phases

PREPARE the statement with variables represented by placeholders

SET Declare variables in the statement

EXECUTE Pass variables to statement and run it

(DROP|DEALLOCATE PREPARE) Remove the prepared statement if no longer needed

Prepared Statements in PHP

PREPARE Statement:

```
1  $sql = "SELECT Name, Avg_Grade FROM Students WHERE StudentId =:id;";  
2  $statement = $pdo->prepare($sql);
```

Placeholder for variable in statement



Prepared Statements in PHP

PREPARE Statement:

```
1 $sql = "SELECT Name, Avg_Grade FROM Students WHERE StudentId =:id;";  
2 $statement = $pdo->prepare($sql);
```

Placeholder for variable in statement



SET Variables:

```
1 $id=1;  
2 $statement->bindParam(':id', $id);
```

Only variables can be provided no values!



Prepared Statements in PHP contd.

EXECUTE Statement:

```
1  $statement->execute();  
2  $result = $statement->fetchAll();
```

Results of statement must be fetched.



Prepared Statements in PHP contd.

EXECUTE Statement:

```
1  $statement->execute();  
2  $result = $statement->fetchAll();
```



Results of statement must be fetched.

DEALLOCATE PREPARED Statement:

- Does not exist in PHP!
- Can be done as regular SQL Query **DEALLOCATE PREPARE stmt;**
- In most databases exist on connection level. On disconnect, statements are removed.

Prepared Statements

```
1  # PREPARE statement
2  $sql = "SELECT Name, Avg_Grade FROM Students WHERE StudentId =:id;";
3  $statement = $pdo->prepare($sql);
4  # SET variables
5  $id = $_GET['id'];
6  $statement->bindParam(':id', $id);
7  # EXECUTE statement
8  $statement->execute();
9  # PROCESS results
10 $result = $statement->fetchAll();
11 foreach ($result as $row) {
12     echo $row['Name'] . " " . $row['Avg_Grade'] . "<br />";
13 }
```

Data Access Objects (DAO)

- Are the core of a Data Access Layer (DAL)
- Implements the Create, Read, Update, Delete (CRUD) pattern
- Makes database logic reusable in your application
- Easily maintainable
- Easily tested
- Improved Security by using prepared statements and parameterized queries

Data Access Object (DAO) in PHP

```
1  interface StudentDAOInterface {  
2      public function createStudent(Student $student): bool; # C  
3  
4      public function getStudentByID(int $studentID): ?Student; # R  
5  
6      public function getAllStudents(): array; # R  
7  
8      public function updateStudent(Student $student): bool; # U  
9  
10     public function deleteStudent(int $studentID): bool; # D  
11 }
```

Create a new record

```
1  // in DAO constructor:
2  $sql = 'INSERT INTO Students (StudentID, Enrolment, Name, Avg_Grade,
        Department) VALUES (:studentID, :enrolment, :name, :avgGrade, :
        department)';
3  $this->create = $this->pdo->prepare($sql);
4
5  // in createStudent:
6  $this->create->bindParam(':studentID', $student->studentID);
7  ...
8  return $this->create->execute(); # returns true on success, false if record
        can't be created
```

Bulk create new records

```
1  $students = array(  
2      array(StudentID => 1, "name" => "Hermione", ...),  
3      array(StudentID => 2, "name" => "Luke", ...), ...  
4  );  
5  $sql = 'INSERT INTO Students (StudentID, Enrolment, Name, Avg_Grade,  
        Department) VALUES (:studentID, :enrolment, :name, :avgGrade, :  
        department)';  
6  $create = $this->pdo->prepare($sql);  
7  foreach ($students as $s) {  
8      $create->execute(  
9          ':studentID' -> $s["StudentID"]  
10         ...  
11     )  
12 }
```

Read single record

```
1  // in DAO constructor:
2  $sql = 'SELECT * FROM Students WHERE StudentID = :studentID';
3  $this->read = $this->pdo->prepare($sql);
4
5  // in getStudentByID:
6  $this->read->bindParam(':studentID', $student->studentID);
7  $result = $stmt->fetch();
8  if ($result) {
9      return new Student($result['StudentID'], ...);
10 } else {
11     return null; # query returns nothing if student id does not exist
12 }
```

Read multiple records

```
1  // in DAO constructor:
2  $sql = 'SELECT * FROM Students';
3  $this->readAll = $this->pdo->prepare($sql);
4
5  // in getAllStudents:
6  $students = [];
7  $result = $this->readAll->fetchAll();
8  foreach ($result as $row) {
9      $students[] = new Student($row['StudentID'], ...);
10 }
11 return $students;
```

fetch vs fetchAll

fetchAll

- Loads ALL records into memory at once.
- Faster than fetch

fetch

- Loads ONE record into memory (one after the other)
- More memory efficient

fetch can replace fetchAll:

```
1  while ($row = $this->readAll->fetch()) {  
2      $students[] = new Student($row['StudentID'], ...);  
3  }
```

Update record

```
1  // in DAO constructor:
2  $sql = 'UPDATE Students SET Enrolment = :enrolment, Name = :name, Avg_Grade
        = :avgGrade, Department = :department WHERE StudentID = :studentID';
3  $this->update = $this->pdo->prepare($sql);
4
5  // in updateStudent:
6  $this->update->bindParam(':studentID', $student->studentID);
7  ...
8  return $stmt->execute(); # returns true on success, false if record can't
    be updated
```

Delete record

```
1  // in DAO constructor:
2  $sql = 'DELETE FROM Students WHERE StudentID = :studentID';
3  $this->delete = $this->pdo->prepare($sql);
4
5  // in deleteStudent:
6  $this->delete->bindParam(':studentID', $student->studentID);
7  return $stmt->execute(); # returns true on success, false if record can't
    be deleted
```


PDO Options

```
1  $host = "localhost";
2  ...
3
4  $options = [
5      PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
6      PDO::MYSQL_ATTR_INIT_COMMAND => "SET NAMES utf8",
7      PDO::ATTR_PERSISTENT => true,
8      PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
9  ];
10 $pdo = new PDO($dsn, $username, $password, $options);
```

Options defined for PDO



Error Mode

Defines strategies for exception handling:

`PDO::ERRMODE_SILENT` Nothing happens on error. Objects have **`errorCode()`** and **`errorInfo()`** to check exceptions.

`PDO::ERRMODE_WARNING` PHP-E_WARNING given

`PDO::ERRMODE_EXCEPTION` A PDOException is raised. Default since PHP 8.0

Fetch Mode

Defines how fetch and fetchAll function.

PDO:FETCH_ASSOC returns associative array

PDO:FETCH_NUM returns indexed array

PDO:FETCH_BOTH (default) returns array that is associative and indexed

PDO:FETCH_OBJ returns anonymous object

others several more modes exist

Other options

`MYSQL_ATTR_INIT_COMMAND SET NAMES utf8` is used to ensure that characters from UTF like ö,ä,ü are handled.

`PDO:ATTR_PERSISTENT` is a performance optimization. Database connections are re-used for different browser sessions

others many options available

Java Database Connectivity (JDBC)

- Standard Java API for connecting to relational databases
- Driver-based: a database-specific JAR provides the **Driver** implementation
- Works with many databases (MySQL, PostgreSQL, SQLite, ...)
- Part of the Java SE standard library (**java.sql.***)
- Same concept as PHP PDO — driver abstraction over raw SQL

JDBC URL

A JDBC URL describes the connection to a database

`jdbc:mysql://localhost:3306/TheDatabase`

`jdbc` JDBC protocol prefix

`mysql` Subprotocol / Driver

`localhost` Host / Server

`3306` Port

`TheDatabase` Database Name

Creating a JDBC Connection

```
1  String url  = "jdbc:mysql://localhost:3306/TheDatabase";
2  String user = "dbuser";
3  String pass = "password";
4  try {
5      Connection conn =
6          DriverManager.getConnection(url, user, pass);
7      // conn is now ready for queries
8  } catch (SQLException e) {
9      System.err.println("Connection failed: " + e.getMessage());
10 }
```

- Modern JDBC 4.0+ auto-loads the driver — no **Class.forName()** needed
- **SQLException** is a checked exception — must be caught or declared
- Use **try-with-resources** to close **Connection** automatically

Prepared Statements in Java — PREPARE

Same three-phase concept as in PHP: **PREPARE SET EXECUTE**

PREPARE Statement:

```
1  String sql = "SELECT Name, Avg_Grade FROM Students "  
2      + "WHERE StudentID = ?"; // <-- positional placeholder  
3  PreparedStatement stmt = conn.prepareStatement(sql);
```

Positional vs Named Placeholders

Java JDBC uses **?** (positional) — **not** named like **:id** in PHP PDO. Index starts at **1**, not 0.

Prepared Statements in Java — SET & EXECUTE

SET Variable (positional, index starts at 1):

```
1    stmt.setInt(1, studentID); // 1 = index of first ?
```

Typed setters: **setString**, **setDouble**, **setBoolean**, ...

Prepared Statements in Java — SET & EXECUTE

SET Variable (positional, index starts at 1):

```
1    stmt.setInt(1, studentID); // 1 = index of first ?
```

Typed setters: `setString`, `setDouble`, `setBoolean`, ...

EXECUTE Statement:

```
1    // SELECT -> returns a ResultSet
2    ResultSet rs = stmt.executeQuery();
3
4    // INSERT / UPDATE / DELETE -> returns rows affected
5    int rowsAffected = stmt.executeUpdate();
```

Prepared Statements in Java — Full Example

```
1  String sql = "SELECT Name, Avg_Grade FROM Students "  
2      + "WHERE StudentID = ?";  
3  PreparedStatement stmt = conn.prepareStatement(sql);  
4  
5  // SET  
6  int id = Integer.parseInt(request.getParameter("id"));  
7  stmt.setInt(1, id);  
8  
9  // EXECUTE  
10 ResultSet rs = stmt.executeQuery();  
11  
12 // PROCESS  
13 while (rs.next()) {  
14     String name      = rs.getString("Name");  
15     double avgGrade = rs.getDouble("Avg_Grade");  
16     System.out.println(name + " " + avgGrade);  
17 }
```

Data Mapping — Student POJO

- PHP returns associative arrays — Java needs an explicit model class
- A **Plain Old Java Object (POJO)** holds one database row

```
1  public class Student {
2      private int    studentID;
3      private String name;
4      private double avgGrade;
5      private String department;
6
7      public Student(int studentID, String name,
8                     double avgGrade, String department) {
9          this.studentID = studentID;
10         this.name      = name;
11         this.avgGrade  = avgGrade;
12         this.department = department;
13     }
14
15     public int    getStudentID() { return studentID; }
16     public String getName()      { return name; }
17     public double getAvgGrade()  { return avgGrade; }
18     public String getDepartment() { return department; }
19 }
```

Data Access Object (DAO) Interface in Java

```
1  import java.sql.SQLException;
2  import java.util.List;
3
4  public interface StudentDAO {
5
6      boolean createStudent(Student student);    // C
7
8      Student getStudentByID(int studentID);    // R
9
10     List<Student> getAllStudents();            // R
11
12     boolean updateStudent(Student student);    // U
13
14     boolean deleteStudent(int studentID);      // D
15 }
```

Create a New Record (INSERT)

```
1  // Prepared once (e.g. in constructor or as field)
2  private static final String INSERT_SQL =
3      "INSERT INTO Students "
4      + "(StudentID, Name, Avg_Grade, Department) "
5      + "VALUES (?, ?, ?, ?)";
6
7  public boolean createStudent(Student s) throws SQLException {
8      PreparedStatement stmt = conn.prepareStatement(INSERT_SQL);
9      stmt.setInt(1, s.getStudentID());
10     stmt.setString(2, s.getName());
11     stmt.setDouble(3, s.getAvgGrade());
12     stmt.setString(4, s.getDepartment());
13     return stmt.executeUpdate() > 0; // true on success
14 }
```

Read Single Record (SELECT by ID)

```
1  private static final String SELECT_ONE =
2      "SELECT * FROM Students WHERE StudentID = ?";
3
4  public Student getStudentByID(int studentID)
5      throws SQLException {
6      PreparedStatement stmt = conn.prepareStatement(SELECT_ONE);
7      stmt.setInt(1, studentID);
8      ResultSet rs = stmt.executeQuery();
9
10     if (rs.next()) {
11         return new Student(
12             rs.getInt("StudentID"),
13             rs.getString("Name"),
14             rs.getDouble("Avg_Grade"),
15             rs.getString("Department")
16         );
17     }
18     return null; // no record found for this ID
19 }
```

Read Multiple Records (SELECT ALL)

```
1  private static final String SELECT_ALL = "SELECT * FROM Students";
2
3  public List<Student> getAllStudents() throws SQLException {
4      PreparedStatement stmt = conn.prepareStatement(SELECT_ALL);
5      ResultSet rs = stmt.executeQuery();
6
7      List<Student> students = new ArrayList<>();
8      while (rs.next()) {                // like PHP fetch()
9          students.add(new Student(
10              rs.getInt("StudentID"),
11              rs.getString("Name"),
12              rs.getDouble("Avg_Grade"),
13              rs.getString("Department")
14          ));
15      }
16      return students;                    // like PHP fetchAll()
17  }
```


Update a Record (UPDATE)

```
1  private static final String UPDATE_SQL =
2      "UPDATE Students "
3      + "SET Name = ?, Avg_Grade = ?, Department = ? "
4      + "WHERE StudentID = ?";
5
6  public boolean updateStudent(Student s) throws SQLException {
7      PreparedStatement stmt = conn.prepareStatement(UPDATE_SQL);
8      stmt.setString(1, s.getName());
9      stmt.setDouble(2, s.getAvgGrade());
10     stmt.setString(3, s.getDepartment());
11     stmt.setInt(4, s.getStudentID());
12     return stmt.executeUpdate() > 0; // true on success
13 }
```

Delete a Record (DELETE)

```
1  private static final String DELETE_SQL =
2      "DELETE FROM Students WHERE StudentID = ?";
3
4  public boolean deleteStudent(int studentID)
5      throws SQLException {
6      PreparedStatement stmt = conn.prepareStatement(DELETE_SQL);
7      stmt.setInt(1, studentID);
8      return stmt.executeUpdate() > 0; // true on success
9  }
```